

Introduction to PIC Programming

Programming Mid-Range PICs in C

by David Meiklejohn, Gooligum Electronics

Lesson 2: Using Timer 0

As we saw in the [previous lesson](#), C can be a viable choice for programming digital I/O operations on mid-range (14-bit) PICs, although we also saw that programs written in C can consume significantly more memory (a limited resource on these tiny MCUs) than equivalent programs written in assembler.

This lesson revisits the material from [mid-range assembler lesson 4](#) on the Timer0 module: using it to time events, to maintain the timing of a background task, for switch debouncing, and as a counter.

Selected examples are re-implemented using Microchip's XC8 compiler¹ (running in "Free mode"), introduced in [lesson 1](#). We'll also see the C equivalents of some of the features covered in [mid-range assembler lesson 5](#), including macros.

In summary, this lesson covers:

- Configuring Timer0 as a timer or counter
- Accessing Timer0
- Using Timer0 for switch debouncing
- Using C macros

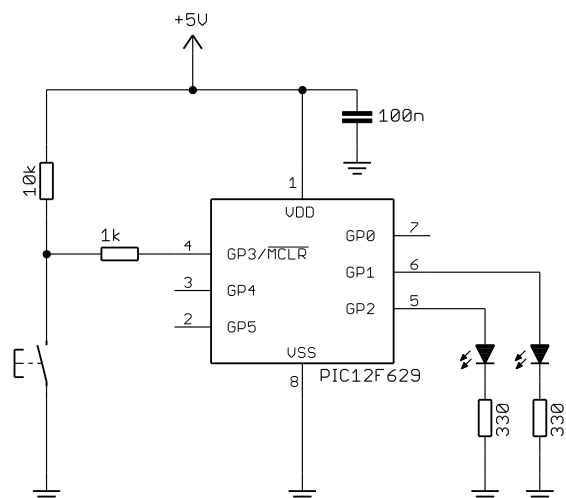
Note that this tutorial series assumes a working knowledge of the C language; it does **not** attempt to teach C.

Example 1: Using Timer0 as an Event Timer

To demonstrate how Timer0 can be used to measure elapsed time, [mid-range assembler lesson 4](#) included a "reaction timer" game, using the circuit on the right.

To implement this circuit using the [Gooligum training board](#), connect jumpers JP3, JP12 and JP13 to enable the pull-up resistor on GP3 and the LEDs on GP1 and GP2.

If you are using Microchip's Low Pin Count Demo Board, you will need to connect LEDs to GP1 and GP2, as described in [baseline assembler lesson 1](#).



¹ Available as a free download from www.microchip.com.

The pushbutton has to be pressed as quickly as possible after the LED on GP2, indicating ‘start’ is lit. If the button is pressed within a predefined reaction time, the LED on GP1 is lit, to indicate ‘success’. Thus, we need to measure the elapsed time between indicating ‘start’ and detecting a pushbutton press.

An ideal way to do that is to use Timer0, in its timer mode (clocked by the PIC’s instruction clock, which in this example is 1 MHz).

Ideally, to make a better “game”, the delay before the ‘start’ LED is lit would be random, but in this simple example, a fixed delay is used.

The program flow can be illustrated in pseudo-code as:

```
do forever
    clear both LEDs
    delay 2 sec
    indicate start
    clear timer
    wait up to 1 sec for button press
    if button pressed and elapsed time < 200ms
        indicate success
    delay 1 sec
end
```

To use Timer0 to measure the elapsed time, we need to extend its range (normally limited to 65 ms) by adding a counter variable, which is incremented each time the timer overflows (or reaches a certain value). In the example in [mid-range lesson 4](#), Timer0 is configured so that it is clocked every 32 μ s, by using the 1 MHz instruction clock with a 1:32 prescaler. After 250 counts, 8 ms ($250 \times 32 \mu$ s) will have elapsed; this is used to increment a counter, which effectively measures time in 8 ms intervals. When the button is pressed, this “8 ms counter” can then be checked, to see whether the maximum reaction time has been exceeded.

As explained in [mid-range lesson 4](#), to select timer mode, with a 1:32 prescaler, we must clear the T0CS and PSA bits, in the OPTION register, and set the PS<2:0> bits to 100. This was done by:

```
movlw    b'11000100'    ; configure Timer0:
                    ; --0-----    timer mode (T0CS = 0)
                    ; ----0----    prescaler assigned to Timer0 (PSA = 0)
                    ; -----100    prescale = 32 (PS = 100)
banksel  OPTION_REG      ;
movwf    OPTION_REG      ; -> increment TMR0 every 32us
```

Here is the main assembler code we had used to implement the button press / timing test routine:

```
        ; wait up to 1 sec for button press
wait1s  clrfs    cnt_8ms          ; clear timer (8 ms counter)
        ; repeat for 1 sec:
banksel TMR0
clrfs    TMR0                  ; clear Timer0
w_tmr0  ; repeat for 8 ms:
banksel GPIO
btfss    BUTTON                ; if button pressed (low)
goto     wait1s_end            ; finish delay loop immediately
banksel  TMR0
movfs    TMR0,w                ;
xorlw    8000/32               ; (8 ms at 32 us/tick)
btfss    STATUS,Z
goto     w_tmr0
incfs    cnt_8ms,f              ; increment 8 ms counter
movlw    1000/8                ; (1 sec at 8 ms/count)
xorwf    cnt_8ms,w
btfss    STATUS,Z
goto     wait1s
wait1s_end
```

```

; indicate success if elapsed time < 200 ms
movlw    MAXRT/8                ; if time < max reaction time (8 ms/count)
subwf    cnt_8ms,w
banksel  GPIO
btfss    STATUS,C
bsf      SUCCESS                ; turn on success LED

```

(This code is actually taken from [mid-range assembler lesson 5](#))

XC8 implementation

Loading the **OPTION** register in XC8 can be done by assigning a value to the variable **OPTION_REG**:

```

OPTION_REG = 0b11000100;          // configure Timer0:
//--0-----          timer mode (T0CS = 0)
//----0---          prescaler assigned to Timer0 (PSA = 0)
//-----100        prescale = 32 (PS = 100)
//                  -> increment every 32 us

```

Note that this has been commented in a way which documents which bits affect each setting, with ‘-’s indicating “don’t care”. For example, we could have instead used ‘**OPTION_REG** = 0b11010100’, since the value of bit 4, or **T0SE**, is irrelevant in timer mode.

However, as we’ve seen, XC8 also makes the individual **OPTION** register bits available as bit-fields defined in the processor header files (such as “pic12f629.h”), so we can instead write this as:

```

// configure Timer0
OPTION_REGbits.T0CS = 0;          // select timer mode
OPTION_REGbits.PSA = 0;          // assign prescaler to Timer0
OPTION_REGbits.PS = 0b100;       // prescale = 32
// -> increment every 32 us

```

Note that, although the **PS** bits can be accessed as the single-bit fields **PS0** to **PS2**, they are also (much more conveniently) made available through a 3-bit field named **PS**, as used here.

This is clear and easy to maintain, and less error-prone, because it is easy to mistype a numeric constant, and the compiler cannot warn you if that happens.

On the other hand, a series of single-bit assignments like this requires more program memory than a whole-register assignment. It is also no longer an *atomic* operation, where all the bits are updated at once. This can be an important consideration in some instances², but it is not relevant here. Note also that the remaining bits in the **OPTION** register are not being explicitly set or cleared; that is ok because in this example we don’t care what values they have.

Which method you use is largely a question of personal style – and you can adapt your style as appropriate. It is often preferable to use symbolic bit names to specify just one or two register bits, but using binary constants if several bits need to be specified at once, especially where some bits need to be set and others cleared (as is the case here), is quite acceptable – assuming that it is clearly commented, as above.

² for example, an interrupt service routine (see [lesson 3](#)) may rely on a peripheral, such as Timer0 here, being fully configured. It would be a problem if the interrupt occurred in the middle of configuration statements like this, with the peripheral only partially configured. These problems can be addressed by selectively disabling and re-enabling interrupts; we’ll see an example of this in [lesson 8](#).

The TMR0 register is accessed through a variable, TMR0, so to clear it, we can write:

```
TMR0 = 0;                // clear timer0
```

and then to wait until 8 ms has elapsed:

```
while (TMR0 < 8000/32)  // wait for 8 ms (32 us/tick)
    ;
```

The “wait for button press or one second” routine can then be implemented as:

```
cnt_8ms = 0;
while (BUTTON == 1 && cnt_8ms < 1000/8)
{
    TMR0 = 0;                // clear timer0
    while (TMR0 < 8000/32)  // wait for 8 ms (32 us/tick)
        ;
    ++cnt_8ms;              // increment 8 ms counter
}
```

where ‘BUTTON’ has been defined as a symbol for ‘GPIObits.GP3’.

As discussed in [mid-range assembler lesson 5](#), your code will be easier to understand and maintain if you use symbolic names to refer to pins. If your design changes, you can update the definitions in one place (usually placed at the start of your c, or in a header file). Of course, you may also need to modify your initialisation statements, such as ‘TRISIO =’. This is a good reason to keep all your initialisation code in one easily-found place, such as at the start of the program, or in an “init()” function.

Finally, checking elapsed time is simply:

```
if (cnt_8ms < MAXRT/8)    // if time < max reaction time (8 ms/count)
    SUCCESS = 1;         // turn on success LED
```

Complete program

Here is the complete reaction timer program, so that you can see how the various parts fit together:

```

/*****
*
* Description:      Lesson 2, example 1
*                  Reaction Timer game.
*
* User must attempt to press button within defined reaction time
* after "start" LED lights. Success is indicated by "success" LED.
*
* Starts with both LEDs unlit.
* 2 sec delay before lighting "start"
* Waits up to 1 sec for button press
* (only) on button press, lights "success"
* 1 sec delay before repeating from start
*
*****
*
* Pin assignments:
* GP1 = success LED
* GP2 = start LED
* GP3 = pushbutton switch (active low)
*
*****/

```

```

#include <xc.h>
#include <stdint.h>

#define _XTAL_FREQ 4000000 // oscillator frequency for _delay()

/***** CONFIGURATION *****/
// int reset, no code protect, no brownout detect, no watchdog,
// power-up timer enabled, int RC clock
__CONFIG(MCLRE_OFF & CP_OFF & CPD_OFF & BOREN_OFF & WDTE_OFF &
        PWRTE_OFF & FOSC_INTRCIO);

// Pin assignments
#define START GPIObits.GP2 // LEDs
#define SUCCESS GPIObits.GP1

#define BUTTON GPIObits.GP3 // pushbutton

/***** CONSTANTS *****/
#define MAXRT 200 // Maximum reaction time in ms

/***** MAIN PROGRAM *****/
void main()
{
    uint8_t cnt_8ms; // counter: increments every 8 ms

    //*** Initialisation

    // configure port
    TRISIO = 0b111001; // configure GP1 and GP2 as outputs

    // configure Timer0
    OPTION_REGbits.T0CS = 0; // select timer mode
    OPTION_REGbits.PSA = 0; // assign prescaler to Timer0
    OPTION_REGbits.PS = 0b100; // prescale = 32
    // -> increment every 32 us

    //*** Main loop
    for (;;)
    {
        // start with both LEDs off
        GPIO = 0;

        // delay 2 sec
        __delay_ms(2000); // delay 2000 ms

        // indicate start
        START = 1; // turn on start LED

        // wait up to 1 sec for button press
        cnt_8ms = 0;
        while (BUTTON == 1 && cnt_8ms < 1000/8)
        {
            TMR0 = 0; // clear timer0
            while (TMR0 < 8000/32) // wait for 8 ms (32 us/tick)
            {
                ++cnt_8ms; // increment 8 ms counter
            }
        }
    }
}

```

```

    // check elapsed time
    if (cnt_8ms < MAXRT/8)      // if time < max reaction time (8 ms/count)
        SUCCESS = 1;           // turn on success LED

    // delay 1 sec
    __delay_ms(1000);           // delay 1000 ms

} // repeat forever
}

```

Comparisons

As we did in [lesson 1](#), we can compare the length of the source code (ignoring comments and white space) versus program and data memory utilisation for this C version with the assembly version from [mid-range lesson 5](#). The stats for the baseline (PIC12F509) versions of this example, from [baseline C lesson 3](#), are also given for comparison:

Reaction_timer

Assembler / Compiler	Source code (lines)		Program memory (words)		Data memory (bytes)	
	12F629	12F509	12F629	12F509	12F629	12F509
Microchip MPASM	58	53	53	55	4	4
XC8 (Free mode)	26	23	87	83	4	4

As expected, the C source code is less than half as long as the assembler source, but the generated C program code is significantly larger (not surprising, given that XC8 does not perform any optimisation when running in “Free mode”).

Example 2: Background Process Timing

As discussed in [mid-range lesson 4](#), one of the key uses of timers is to provide regular timing for “background” processes, while a “foreground” process responds to user signals. Timers are ideal for this, because they continue to run, at a steady rate, regardless of any processing the PIC is doing. On mid-range PICs this is normally done using timer-driven interrupts, which will be covered in the [next lesson](#). However, the non-interrupt method, described in [baseline C lesson 3](#), can still be used, and is covered here mainly for completeness.

The example in [mid-range lesson 4](#) used the circuit above, flashing the LED on GP2 at a steady 1 Hz, while lighting the LED on GP1 whenever the pushbutton is pressed.

The 500 ms delay needed for the 1 Hz flash was derived from Timer0 as follows:

- Using a 4 MHz processor clock, providing a 1 MHz instruction clock and a 1 μ s instruction cycle
- Assigning a 1:32 prescaler to the instruction clock, incrementing Timer0 every 32 μ s
- Resetting Timer0 to zero, as soon as it reaches 125 (i.e. every $125 \times 32 \mu\text{s} = 4 \text{ ms}$)
- Repeating 125 times, creating a delay of $125 \times 4 \text{ ms} = 500 \text{ ms}$.

This was implemented by the following code:

```
main_loop
    ; delay 500 ms
    movlw    .125                ; repeat 125 times (125 x 4 ms = 500 ms)
    movwf    dly_cnt

dly500
    banksel  TMR0                ; clear timer0
    clrf     TMR0
w_tmr0     movf     TMR0,w        ; wait for 4 ms
    xorlw    .125                ; (125 ticks x 32 us/tick = 4 ms)
    btfss    STATUS,Z
    goto     w_tmr0
    decfsz   dly_cnt,f          ; end 500 ms delay loop
    goto     dly500

    ; toggle flashing LED
    movf     sGPIO,w
    xorlw    1<<GP2            ; toggle LED on GP2
    movwf    sGPIO              ; using shadow register
    banksel  GPIO
    movwf    GPIO

    ; repeat forever
    goto     main_loop
```

And then the code which responds to the pushbutton was placed within the timer wait loop:

```
w_tmr0
    banksel  GPIO                ; repeat for 4 ms:
    bcf      sGPIO,GP1           ; check and respond to button press
    btfss    GPIO,GP3           ; assume button up -> indicator LED off
    bsf      sGPIO,GP1          ; if button pressed (GP3 low)
    movf     sGPIO,w            ; turn on indicator LED
    movwf    GPIO               ; update port (copy shadow to GPIO)
    banksel  TMR0
    movf     TMR0,w
    xorlw    .125                ; (125 ticks x 32 us/tick = 4 ms)
    btfss    STATUS,Z
    goto     w_tmr0
```

The additional code doesn't affect the timing of the background task (flashing the LED), because there are only a few additional instructions; they are able to be executed within the 32 μ s available between each "tick" of Timer0.

XC8 implementation

There are no new features to introduce; Timer0 is setup and accessed in the same way as in the last example. The assembly code above can be implemented with XC8 as:

```
for (;;)
{
    // delay 500 ms while responding to button press
    for (dc = 0; dc < 125; dc++)    // repeat 125 times (125 x 4 ms = 500 ms)
    {
        TMR0 = 0;                    // clear timer0
        while (TMR0 < 125)           // repeat for 4 ms (125 x 32 us)
        {
            // check and respond to button press
            sGPIO &= ~(1<<1);        // assume button up -> LED off
            if (GP3 == 0)             // if button pressed (GP3 low)
                sGPIO |= 1<<1;       // turn on LED on GP1
            GPIO = sGPIO;             // update port (copy shadow to GPIO)
        }
        // toggle flashing LED
        sGPIO ^= 1<<2;               // toggle LED on GP2 using shadow reg
    }
} // repeat forever
```

There is no need to update GPIO after the LED on GP2 is toggled, because GPIO is being continually updated from sGPIO within the inner timer wait loop.

Note the syntax used to set, clear and toggle bits in the shadow GPIO variable, sGPIO:

```
sGPIO |= 1<<1;           // turn on LED on GP1
sGPIO &= ~(1<<1);        // turn off LED on GP1
sGPIO ^= 1<<2;           // toggle LED on GP2
```

We could instead have written:

```
sGPIO |= 0b000010;       // turn on LED on GP1
sGPIO &= 0b111101;       // turn off LED on GP1
sGPIO ^= 0b000100;       // toggle LED on GP2
```

But the left shift ('<<') form more clearly specifies which bit is being operated on.

If we define symbols representing the port bit positions:

```
#define nFLASH  2          // flashing LED on GP2
#define nPRESS  1          // "button pressed" indicator LED on GP1
```

we can write these statements as:

```
sGPIO |= 1<<nPRESS;       // turn on indicator LED
sGPIO &= ~(1<<nPRESS);    // turn off indicator LED
sGPIO ^= 1<<nFLASH;       // toggle flashing LED
```

These symbols can also be used when configuring the port directions:

```
TRISIO = ~(1<<nFLASH|1<<nPRESS); // configure LEDs (only) as outputs
```

This makes the code clearer, more general, and therefore more maintainable.

However, this approach doesn't work well on bigger PICs, which have more than one port. You still need to keep track of which port each pin belongs to, and if you change your pin assignments later, you may well need to make a number of changes throughout your code.

A more robust approach is to make use of *bitfields* within C structures.

For example:

```
struct {
    unsigned    GP0      : 1;
    unsigned    GP1      : 1;
    unsigned    GP2      : 1;
    unsigned    GP3      : 1;
    unsigned    GP4      : 1;
    unsigned    GP5      : 1;
} sGPIObits;
```

It is then possible to refer to each bit as a structure member, for example:

```
sGPIObits.GP1 = 1;
```

and if we also defined a symbol such as:

```
#define SPRESS    sGPIObits.GP1
```

we can then write this as:

```
SPRESS = 1;
```

That's nice – we have “shadow bits” and we can refer to them easily by symbolic names – but there's still a problem. As well as being able to access individual bits, we also need to be able to refer to the whole shadow register as a single variable, to read or update all the bits at once. After all, that's the whole point of using a shadow register.

We want to be able to change a single bit, as in:

```
sGPIObits.GP1 = 1;    // set shadow GP1
```

and also read the whole shadow register in a single operation, as in:

```
GPIO = sGPIO;          // copy shadow register to port
```

How can we do both?

The C *union* construct is intended for exactly this situation, where we need to access the memory holding a variable in more than one way.

We can define for example:

```
union {
    uint8_t      port;                // shadow copy of GPIO
    struct {
        unsigned    GP0      : 1;
        unsigned    GP1      : 1;
        unsigned    GP2      : 1;
        unsigned    GP3      : 1;
        unsigned    GP4      : 1;
        unsigned    GP5      : 1;
    };
} sGPIO;
```

This allows us to refer to the shadow register as `sGPIO.port`, representing the whole port, in a single operation. For example:

```
sGPIO.port = 0;          // clear shadow register

GPIO = sGPIO.port;      // update port (copy shadow to GPIO)
```

We can also refer to the individual shadow bits as, for example:

```
sGPIO.GP1 = 1;          // set shadow GP1
```

If we define symbols representing these shadow bits:

```
#define sFLASH  sGPIO.GP2      // flashing LED (shadow)
#define sPRESS  sGPIO.GP1      // "button pressed" indicator LED (shadow)
```

we can rewrite the previous bit-manipulation statements as:

```
sPRESS = 1;                // turn on indicator LED
sPRESS = 0;                // turn off indicator LED
sFLASH = !sFLASH;          // toggle flashing LED
```

and, very concisely:

```
sPRESS = !BUTTON;          // turn on indicator only if button pressed
```

Besides clarity and conciseness, a big advantage of this technique is that, if (on a larger PIC) you were to move one of these functions (such as the flashing LED) to another port, you only need to modify the symbol definition and perhaps your initialisation routine. The rest of your program could stay the same – these statements would still work.

Defining the shadow register as a union incorporating a bitfield structure may seem like a lot of trouble for an apparently small benefit, but it's an elegant approach that will pay off as your applications become more complex.

Complete program

Here is how this shadow register union / bitfield structure definition is used in practice:

```

/*****
 *   Description:      Lesson 2, example 2b
 *
 *   Demonstrates use of Timer0 to maintain timing of background actions
 *   while performing other actions in response to changing inputs
 *
 *   One LED simply flashes at 1 Hz (50% duty cycle).
 *   The other LED is only lit when the pushbutton is pressed
 *
 *   Uses union / bitfield structure to represent shadow register
 *
 *****/
 *   Pin assignments:
 *       GP1 = "button pressed" indicator LED
 *       GP2 = flashing LED
 *       GP3 = pushbutton switch (active low)
 *
 *****/

#include <xc.h>
#include <stdint.h>

```

```

/***** CONFIGURATION *****/
// int reset, no code protect, no brownout detect, no watchdog,
// power-up timer enabled, int RC clock
__CONFIG(MCLRE_OFF & CP_OFF & CPD_OFF & BOREN_OFF & WDTE_OFF &
        PWRTE_OFF & FOSC_INTRCIO);

// Pin assignments
#define sFLASH    sGPIO.GP2        // flashing LED (shadow)
#define sPRESS    sGPIO.GP1        // "button pressed" indicator LED (shadow)
#define BUTTON    GPIObits.GP3     // pushbutton

/***** GLOBAL VARIABLES *****/
union {                                // shadow copy of GPIO
    uint8_t      port;
    struct {
        unsigned GP0      : 1;
        unsigned GP1      : 1;
        unsigned GP2      : 1;
        unsigned GP3      : 1;
        unsigned GP4      : 1;
        unsigned GP5      : 1;
    };
} sGPIO;

/***** MAIN PROGRAM *****/
void main()
{
    uint8_t  dc;                        // delay counter

    //*** Initialisation

    // configure port
    GPIO = 0;                          // start with all LEDs off
    sGPIO.port = 0;                    // update shadow
    TRISIO = ~(1<<1|1<<2);           // configure GP1 and GP2 (only) as outputs

    // configure Timer0
    OPTION_REGbits.T0CS = 0;           // select timer mode
    OPTION_REGbits.PSA = 0;            // assign prescaler to Timer0
    OPTION_REGbits.PS = 0b100;         // prescale = 32
                                        // -> increment every 32 us

    //*** Main loop
    for (;;)
    {
        // delay 500 ms while responding to button press
        for (dc = 0; dc < 125; dc++) // repeat 125 times (125 x 4 ms = 500 ms)
        {
            TMR0 = 0;                 // clear timer0
            while (TMR0 < 125)         // repeat for 4 ms (125 x 32 us)
            {
                sPRESS = !BUTTON;     // turn on LED only if button pressed
                GPIO = sGPIO.port;    // update port (copy shadow to GPIO)
            }
        }
        // toggle flashing LED
        sFLASH = !sFLASH;             // toggle flashing LED (shadow)

    } // repeat forever
}

```

Comparisons

Here is the resource usage summary for the “Flash an LED while responding to a pushbutton” programs (the C version defining the shadow register as a union containing a bitfield structure, as above):

Flash+PB_LED

Assembler / Compiler	Source code (lines)		Program memory (words)		Data memory (bytes)	
	12F629	12F509	12F629	12F509	12F629	12F509
Microchip MPASM	44	37	38	31	2	2
XC8 (Free mode)	31	28	78	72	3	2

The C source code is comparatively long in this example, because of the shadow register union / bitfield structure definition. It’s a big part of the source code – something you wouldn’t normally bother with, for such a small program. But we’ll keep doing it this way, because it’s good practice that will serve us well as our programs become longer, and the extra lines of variable definition won’t seem like such a big deal.

Example 3: Switch debouncing

The [previous lesson](#) demonstrated one method commonly used to debounce switches: sampling the switch state periodically, and only considering it to have definitely changed when it has been in the new state for some minimum number of successive samples.

This “counting algorithm” was given as:

```
count = 0
while count < max_samples
    delay sample_time
    if input = required_state
        count = count + 1
    else
        count = 0
end
```

As explained in [mid-range lesson 4](#), this can be simplified by using a timer, since the timer increments automatically:

```
reset timer
while timer < debounce_time
    if input ≠ required_state
        reset timer
end
```

This algorithm was implemented in assembler, to wait for and debounce a “button down” event, as follows:

```
banksel TMR0
wait_dn clrf TMR0 ; reset timer
chk_dn btfsc GPIO,GP3 ; check for button press (GP3 low)
goto wait_dn ; continue to reset timer until button down
movf TMR0,w ; has 10 ms debounce time elapsed?
xorlw .157 ; (157 = 10ms/64us)
btfss STATUS,Z ; if not, continue checking button
goto chk_dn
```

This code assumes that Timer0 is available, and is in timer mode, with a 1 MHz instruction clock and a 1:64 prescaler, giving 64 μ s per tick.

Although mid-range PICs have more than one timer, they remain a scarce resource, and it is likely that Timer0 is being used for something else, and so is not available for switch debouncing. As we'll see in the [next lesson](#), it can be better to use a regular timer-driven interrupt for switch debouncing, allowing a single timer (driving the interrupt) to be used for a number of tasks.

But if you're not using Timer0 for anything else, using it for switch debouncing is perfectly reasonable.

This technique was demonstrated by applying this timer-based debouncing method to the "toggle an LED on pushbutton press" program developed in [mid-range assembler lesson 3](#).

XC8 implementation

Timer0 can be configured for timer mode, with a 1:64 prescaler, by:

```
OPTION_REGbits.T0CS = 0;           // select timer mode
OPTION_REGbits.PSA = 0;           // assign prescaler to Timer0
OPTION_REGbits.PS = 0b101;       // prescale = 64
                                   // -> increment every 64 us
```

This is the same as for the 1:32 prescaler examples, above, except that the PS<2:0> bits are set to '101' instead of '100'.

The timer-based debounce algorithm, given above in pseudo-code, is readily translated into C:

```
TMR0 = 0;                          // reset timer
while (TMR0 < 157)                 // wait at least 10 ms (157 x 64 us = 10 ms)
    if (GPIObits.GP3 == 1)         // if button up,
        TMR0 = 0;                  // restart wait
```

Using C macros

This fragment of code is one that we might want to use a number of times, perhaps modified to debounce switches on inputs other than GP3, in this or other programs.

As we saw in [mid-range lesson 5](#), the MPASM assembler provides a *macro* facility, which allows a parameterised segment of code to be defined once and then inserted multiple times into the source code.

Macros can also be used when programming in C.

For example, we could define our debounce routine as a macro as follows:

```
#define DEBOUNCE 10*1000/256      // switch debounce count = 10 ms/(256us/tick)

// DbnceLo()
//
// Debounce switch on given input pin
// Waits for switch input to be low continuously for DEBOUNCE*256/1000 ms
//
// Uses: TMR0           Assumes: TMR0 running at 256 us/tick
//
#define DbnceLo(PIN) TMR0 = 0;          /* reset timer          */ \
    while (TMR0 < DEBOUNCE)             /* wait until debounce time */ \
        if (PIN == 1)                   /* if input high,          */ \
            TMR0 = 0;                   /* restart wait            */ \
```

Note that a backslash (`\`) is placed at the end of all but the last line, to continue the macro definition over multiple lines. To make the backslashes visible to the C pre-processor, the older “/* */” style comments must be used, instead of the newer “//” style.

This macro can then be used within your program as, for example:

```
DbtnceLo (GPIObits.GP3);    // wait until button pressed (GP3 low)
```

You can define macros toward the start of your source code, but as you build your own library of useful macros, you would normally keep them together in one or more header files, such as “stdmacros.h”, and reference them from your main program, using the #include directive.

Complete program

Here is how this timer-based debounce code (without using macros) fits into the XC8 version of the “toggle an LED on pushbutton press” program:

```

/*****
 *   Description:      Lesson 2, example 3a
 *
 *   Demonstrates use of Timer0 to implement debounce counting algorithm
 *
 *   Toggles LED when pushbutton is pressed then released
 *
 *****/
 *   Pin assignments:
 *       GP1 = flashing LED
 *       GP3 = pushbutton switch (active low)
 *
 *****/

#include <xc.h>
#include <stdint.h>

/***** CONFIGURATION *****/
// int reset, no code protect, no brownout detect, no watchdog,
// power-up timer enabled, int RC clock
__CONFIG(MCLRE_OFF & CP_OFF & CPD_OFF & BOREN_OFF & WDTE_OFF &
        PWRTE_OFF & FOSC_INTRCIO);

// Pin assignments
#define sFLASH    sGPIO.GP1           // flashing LED (shadow)
#define BUTTON    GPIObits.GP3       // pushbutton

/***** GLOBAL VARIABLES *****/
union {
    uint8_t      port;                // shadow copy of GPIO
    struct {
        unsigned GP0    : 1;
        unsigned GP1    : 1;
        unsigned GP2    : 1;
        unsigned GP3    : 1;
        unsigned GP4    : 1;
        unsigned GP5    : 1;
    };
} sGPIO;

```

```

/***** MAIN PROGRAM *****/
void main()
{
    //*** Initialisation

    // configure port
    GPIO = 0;                                // start with LED off
    sGPIO.port = 0;                          // update shadow
    TRISIO = 0b111101;                      // configure GP1 (only) as an output

    // configure Timer0
    OPTION_REGbits.T0CS = 0;                // select timer mode
    OPTION_REGbits.PSA = 0;                 // assign prescaler to Timer0
    OPTION_REGbits.PS = 0b101;              // prescale = 64
                                           // -> increment every 64 us

    //*** Main loop
    for (;;)
    {
        // wait for button press, debounce using timer0:
        TMR0 = 0;                          // reset timer
        while (TMR0 < 157)                  // wait at least 10 ms (157 x 64 us = 10 ms)
            if (BUTTON == 1)                // if button up,
                TMR0 = 0;                    // restart wait

        // toggle LED
        sFLASH = !sFLASH;                   // toggle flashing LED (shadow)
        GPIO = sGPIO.port;                  // write to GPIO

        // wait for button release, debounce using timer0:
        TMR0 = 0;                          // reset timer
        while (TMR0 < 157)                  // wait at least 10ms (157 x 64us = 10 ms)
            if (BUTTON == 0)                // if button down,
                TMR0 = 0;                    // restart wait

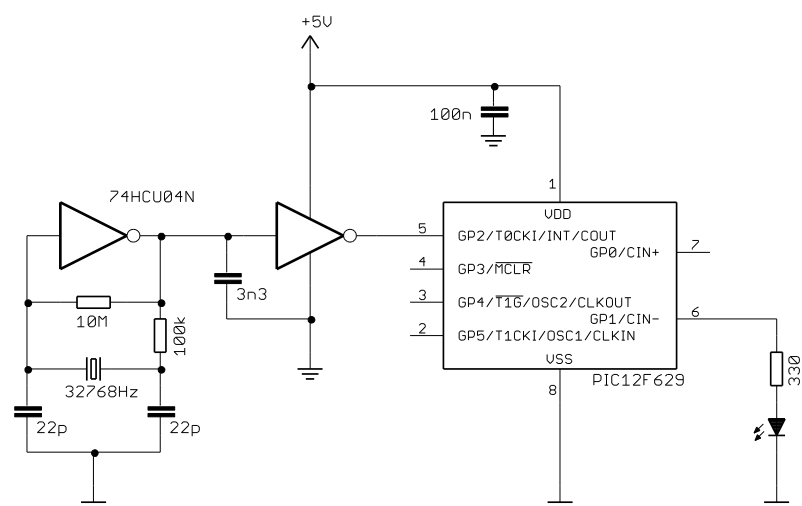
    }    // repeat forever
}

```

Example 4: Using Counter Mode

Until now we've used Timer0 in "timer mode", where it is clocked by the PIC's instruction clock, which runs at one quarter the speed of the processor clock (i.e. 1 MHz when the 4 MHz internal RC oscillator is used). As we saw in [mid-range lesson 4](#), the timer can instead be used in "counter mode", where it counts transitions (rising or falling) on the PIC's T0CKI input.

We can use the example from that lesson to show how Timer0 can be used as a counter, using C: Timer0 is driven by an external 32.768 kHz crystal oscillator (as shown on the right), providing a time base that can be used to flash an LED at a more accurate 1 Hz.



To configure the [Gooligum training board](#) for this example, close jumper JP22 to connect the 32 kHz clock signal to T0CKI, and close jumpers JP3 and JP12 to enable the external $\overline{\text{MCLR}}$ pull-up resistor (not shown in this diagram, for clarity) and the LED on GP1. If you are using Microchip's Low Pin Count Demo Board, you will need to build the oscillator circuit separately, as described in [baseline assembler lesson 5](#).

If the 32.768 kHz clock input is divided (prescaled) by 128, bit 7 of TMR0 will cycle at 1 Hz.

To configure Timer0 for counter mode (external clock on T0CKI) with a 1:128 prescale ratio, we need to set the T0CS bit to '1', PSA to '0' and PS<2:0> to '110'.

This was done in [mid-range assembler lesson 4](#) by:

```
movlw    b'11110110'      ; configure Timer0:
                    ; --1----- counter mode (T0CS = 1)
                    ; ----0--- prescaler assigned to Timer0 (PSA = 0)
                    ; -----110 prescale = 128 (PS = 110)
banksel  OPTION_REG      ; -> increment at 256 Hz with 32.768 kHz input
movwf    OPTION_REG
```

The value of T0SE bit is irrelevant; we don't care if the counter increments on the rising or falling edge of the input clock signal – only the frequency is important. Either edge will do.

Bit 7 of TMR0 (which is cycling at 1 Hz) was then continually copied to GP1 (using a shadow register), as follows:

```
loop      ; transfer TMR0<7> to GP1
    clrf   sGPIO          ; assume TMR0<7>=0 -> LED off
    banksel TMR0
    btfsc  TMR0,7          ; if TMR0<7>=1
    bsf    sGPIO,GP1       ; turn on LED

    movf   sGPIO,w         ; copy shadow to GPIO
    banksel GPIO
    movwf  GPIO

    ; repeat forever
    goto  loop
```

XC8 implementation

Configuring Timer0 is simply:

```
OPTION_REGbits.T0CS = 1;      // select counter mode
OPTION_REGbits.PSA = 0;      // assign prescaler to Timer0
OPTION_REGbits.PS = 0b110;   // prescale = 128
                             // -> incr at 256 Hz with 32.768 kHz input
```

To copy bit 7 of TMR0 to the LED (via a shadow bit), we can use the following construct:

```
sFLASH = 0;                  // assume TMR<7>=0 -> LED off
if (TMR0 & 1<<7)             // if TMR0<7>=1
    sFLASH = 1;              // turn on LED
```

This works because the expression "1<<7" equals 10000000 binary, so the result of ANDing TMR0 with 1<<7 will only be non-zero if TMR0<7> is set.

Or we could write this equivalently as:

```
sFLASH = (TMR0 & 1<<7) != 0;    // sFLASH = TMR0<7>
```


Which construct you use is, as ever, a matter of personal style; we'll use the second version here.

Complete program

Here is the XC8 version of the “flash an LED using crystal-driven timer” program:

```

/*****
 *   Description:      Lesson 2, example 4
 *
 *   Demonstrates use of Timer0 in counter mode
 *
 *   LED flashes at 1 Hz (50% duty cycle),
 *   with timing derived from 32.768 kHz input on T0CKI
 *
 *****/
 *   Pin assignments:
 *   GP1    = flashing LED
 *   T0CKI  = 32.768 kHz signal
 *
 *****/

#include <xc.h>
#include <stdint.h>

/***** CONFIGURATION *****/
// ext reset, no code protect, no brownout detect, no watchdog,
// power-up timer enabled, int RC clock
__CONFIG(MCLRE_ON & CP_OFF & CPD_OFF & BOREN_OFF & WDTE_OFF &
         PWRTE_OFF & FOSC_INTRCIO);

// Pin assignments
#define sFLASH  sGPIO.GP1                // flashing LED (shadow)

/***** GLOBAL VARIABLES *****/
union {                                // shadow copy of GPIO
    uint8_t      port;
    struct {
        unsigned GP0      : 1;
        unsigned GP1      : 1;
        unsigned GP2      : 1;
        unsigned GP3      : 1;
        unsigned GP4      : 1;
        unsigned GP5      : 1;
    };
} sGPIO;

/***** MAIN PROGRAM *****/
void main()
{
    /*** Initialisation

    // configure port
    TRISIO = ~(1<<1);                // configure GP1 (only) as an output

    // configure Timer0
    OPTION_REGbits.T0CS = 1;          // select counter mode
    OPTION_REGbits.PSA = 0;           // assign prescaler to Timer0
    OPTION_REGbits.PS = 0b110;        // prescale = 128
    // -> incr at 256 Hz with 32.768 kHz input
    
```

```
//*** Main loop
for (;;)
{
    // TMR0<7> cycles at 1 Hz, so continually copy to LED
    sFLASH = (TMR0 & 1<<7) != 0;    // sFLASH = TMR0<7>

    GPIO = sGPIO.port;              // copy shadow to GPIO
}  // repeat forever
}
```

Summary

These examples have demonstrated that Timer0 can be effectively configured and accessed using the XC8 compiler, with the program algorithms being able to be expressed quite succinctly in C.

We've also seen that using symbolic names and macros can help make your code more maintainable, and how the union and bitfield structure constructs can be used to make it possible to access both a whole variable and its individual bits, in an elegant way.

In the [next lesson](#) we'll see how interrupts can be implemented using XC8.